Atlas — Technical Architecture & Discovery OS Vision

> **"The AI does not know things. It queries a living knowledge substrate… and reasons over it automatically."**

Version: 1.0.0 | **Date:** February 2026 | **Status:** Production-Ready Core, Discovery Vision in Design

---

Table of Contents

1. System Overview
2. Architecture Deep-Dive
3. The Knowledge Pipeline
4. The Multi-Agent Swarm
5. Hardware Optimization & Constraints
6. Data Model & Storage
7. Configuration & Extensibility
8. The Vision: Discovery OS for Small Molecule & Glycan Research
9. Critical Feasibility Analysis
10. Roadmap

---

1. System Overview

Atlas is a **standalone Windows desktop application** that constructs a continuous, queryable knowledge layer beneath an AI model. It is not a chatbot—it is a local-first, agentic research operating system.

The application is fully self-contained: no cloud accounts, no external databases, no Docker. A single `.msi` installer bundles the entire stack.

What Atlas Does Today

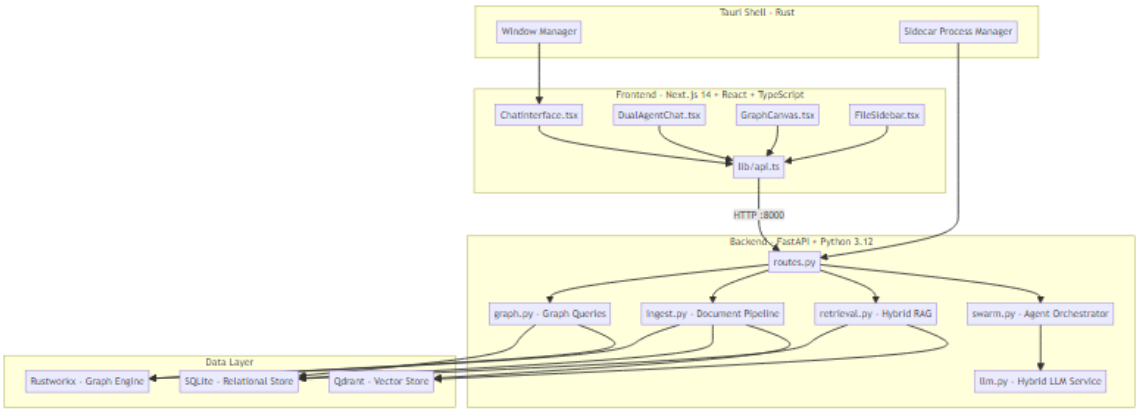
Capability	Implementation
Document Ingestion	PDF, DOCX, TXT → text extraction → semantic chunking → entity extraction → vector embedding
Hybrid RAG	Vector search (Qdrant) + BM25 keyword search + Knowledge Graph traversal, fused via Reciprocal Rank Fusion

Capability	Implementation
Multi-Agent Reasoning	LangGraph-orchestrated swarm with intent routing, reflection loops, and MoE task decomposition
Anti-Hallucination	Grounding Verifier audits every factual claim against source text with tiered confidence badges
Knowledge Graph	Rustworkx-backed entity-relationship graph with centrality analysis and subgraph extraction
Local Inference	`llama-cpp-python` running quantized GGUF models on consumer GPUs (4–8 GB VRAM)
Cloud Hybrid	Optional API fallback to DeepSeek, MiniMax, OpenAI, or Anthropic via LiteLLM

2. Architecture Deep-Dive

2.1 Three-Layer Desktop Stack

Atlas is a Tauri desktop app wrapping a Next.js frontend and a Python FastAPI backend. All three run as local processes; no network egress is required.



2.2 Key File Map

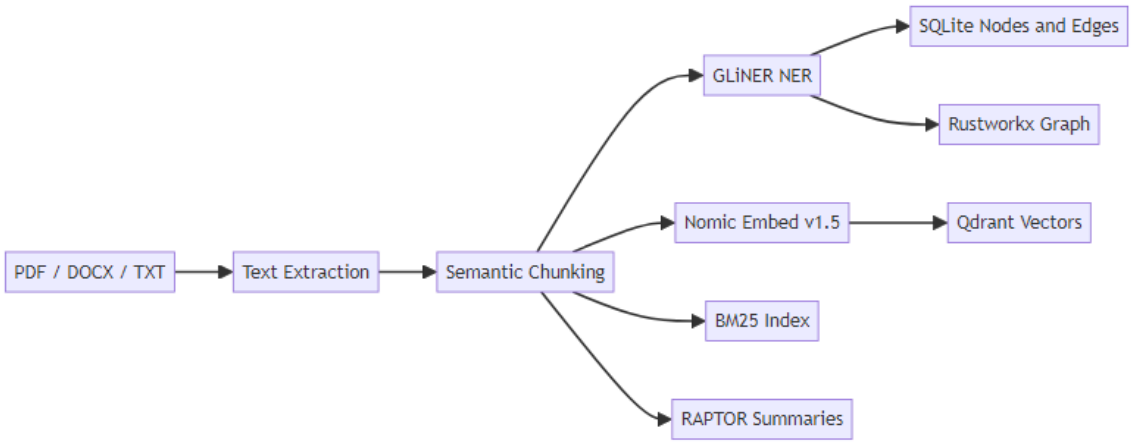
Layer	File	
Backend Core	`src/backend/app/services/swarm.py`	Full a Navig
Backend Core	`src/backend/app/services/llm.py`	Hybri via Li
Backend Core	`src/backend/app/services/retrieval.py`	Hybri Recip

Layer	File	
Backend Core	`src/backend/app/services/ingest.py`	Docu chunk embe
Agents	`src/backend/app/services/agents/meta_router.py`	Intent (SIM + mo
Agents	`src/backend/app/services/agents/librarian.py`	Fast 2 Hand
Agents	`src/backend/app/services/agents/supervisor.py`	MoE deleg
Agents	`src/backend/app/services/agents/grounding.py`	Anti-l verifi
Experts	`src/backend/app/services/agents/experts/hypothesis.py`	Gene evide
Experts	`src/backend/app/services/agents/experts/retrieval_expert.py`	Targe
Experts	`src/backend/app/services/agents/experts/writer.py`	Draft
Experts	`src/backend/app/services/agents/experts/critic.py`	Revie gaps
Config	`src/backend/app/core/config.py`	50+ t iterati
Database	`src/backend/app/core/database.py`	SQL Docu

---

3. The Knowledge Pipeline

When a user uploads a document, Atlas runs a multi-stage ingestion pipeline defined in `ingest.py`:



Key implementation details:

- **Text Extraction:** Tries Docling VLM first for structure-preserving extraction (tables, charts), falls back to `pdfplumber`/`PyPDF` (`_extract_pdf_text` in `ingest.py`)

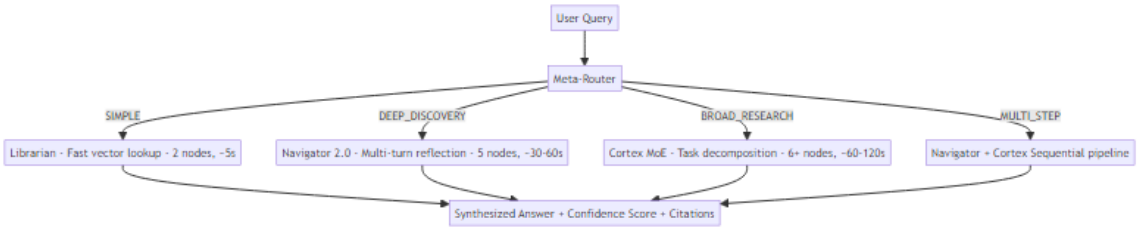
- **Chunking:** Uses semantic chunking via `semantic_chunker.py` (target: 512 tokens), falls back to fixed-size overlap (1000 chars, 200 overlap)
- **Entity Extraction:** GLiNER (`gliner_small-v2.1`, ~50 MB) extracts typed entities (PERSON, ORGANIZATION, CONCEPT, etc.) without requiring an LLM call, via `_extract_entities_gliner`
- **RAPTOR:** `raptor.py` generates hierarchical cluster summaries (L1 clusters) for each document, enabling multi-resolution retrieval
- **BM25:** `bm25_index.py` maintains a sparse keyword index fused with vector results via Reciprocal Rank Fusion (`rrf_fuse`)

---

4. The Multi-Agent Swarm

The core intelligence lives in `swarm.py` (2,787 lines), orchestrated via LangGraph `StateGraph` objects.

4.1 Query Routing



The `Meta-Router` (`meta_router.py`) classifies each query using a zero-shot LLM classification prompt and also performs **dynamic model swapping** — selecting faster 3B models for simple queries and larger 7B–8B models for deep discovery tasks via `ensure_optimal_model`.

4.2 Librarian Agent (Fast Path)

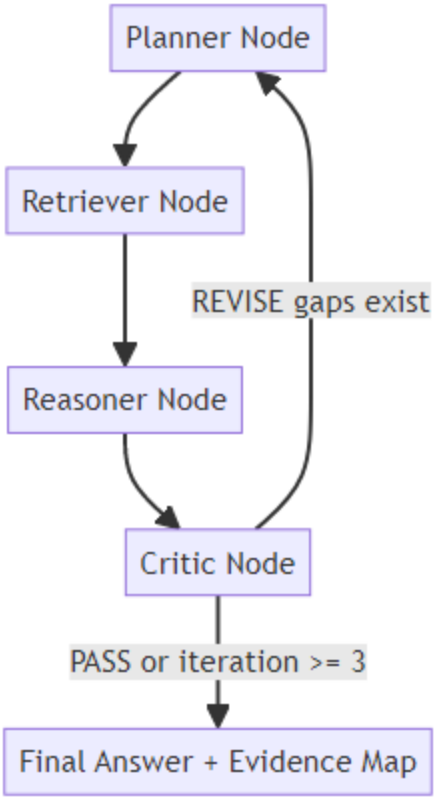
Defined in `librarian.py`. A minimal 2-node LangGraph:

```
retrieve_node → answer_node → END
```

- **Retrieve:** Vector search via Qdrant `query_points` → optional FlashRank reranking → cosine threshold filtering (≥0.4 for vector, ≥0.05 for reranked)
- **Answer:** Single LLM call with XML-structured output (`<reasoning>`, `<confidence>`, `<answer>` tags)
- Handles **~80% of queries** in under 5 seconds

4.3 Navigator 2.0 (Deep Discovery with Reflection)

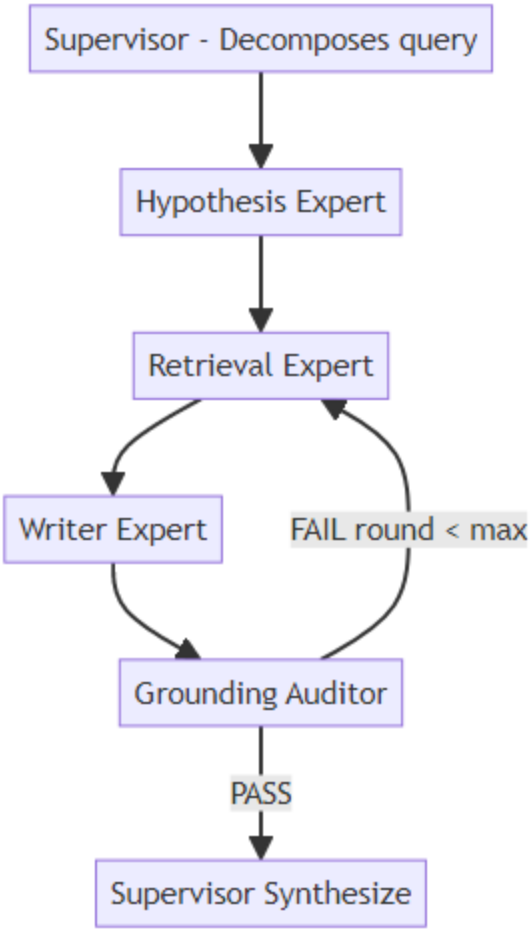
The Navigator implements multi-turn reflection loops (up to `MAX_REFLECTION_ITERATIONS=3`). Its LangGraph contains 5 nodes:



State tracked per iteration (from `NavigatorState` in `swarm.py`): - `reasoning_plan`, `identified_gaps`, `search_terms` — The Planner's output - `graph_summary` — Rustworkx subgraph centrality analysis - `accumulated_evidence` — Grows across reflection iterations - `confidence_score` — Auto-passes at ≥0.75 threshold (`NAVIGATOR_CONFIDENCE_THRESHOLD`) - `iteration_count` — Capped at 3 to prevent infinite loops

4.4 Cortex MoE (Mixture of Experts)

For broad research queries, the Cortex decomposes the question into sub-tasks and delegates to specialized experts. Built in `supervisor.py`:



MoE State (`MoEState` in `supervisor.py`) tracks: - `sub_tasks` — Decomposed sub-queries (default: 5 via `CORTEX_NUM_SUBTASKS`) - `hypotheses` and `selected_hypothesis` — Ranked by the Hypothesis Expert - `draft` and `draft_version` — Iterative drafting by the Writer Expert - `grounding_results`, `ungrounded_claims`, `audit_verdict` — From the Grounding Auditor - `max_rounds` — Capped at 5 (`MOE_MAX_EXPERT_ROUNDS`)

4.5 Grounding Verifier (Anti-Hallucination)

Defined in `grounding.py`. Every factual claim in an agent's output is individually checked:

1. **Claim Extraction:** LLM call decomposes the answer into numbered factual claims
2. **Per-Claim Verification:** Each claim is embedded and searched against Qdrant; cosine similarity determines badge level:
   - `GROUNDED` (score > 0.82) — Directly supported by cited source
   - `SUPPORTED` (0.72–0.82) — Paraphrased but source matches
   - `INFERRED` (0.60–0.72) — Synthesis/inference, not verbatim in source
   - `UNVERIFIED` (< 0.60) — No matching source found
3. **Overall Score:** Percentage of claims that are `GROUNDED` or `SUPPORTED`

---

5. Hardware Optimization & Constraints

Atlas targets the **NVIDIA RTX 3050 (4 GB VRAM)** as the minimum hardware. Every architectural decision flows from this constraint:

Technique	Implementation	Why it Matters
Sequential Agent Execution	LangGraph nodes run serially	Only one LLM inference at a time; never exceeds VRAM
GBNF Grammar Constraints	`generate_with_validation` in `swarm.py`	Forces small 3B–7B models to output valid JSON/XML without fine-tuning
Dynamic Model Swapping	`ensure_optimal_model` in `meta_router.py`	Uses fast 3B models for simple queries, 7B+ for complex ones
Partial GPU Offload	`DEFAULT_GPU_LAYERS = 35` in `llm.py`	Offloads ~3.2 GB to GPU, keeps rest in RAM; tunable via env var
GLiNER NER	`_extract_entities_gliner` in `ingest.py`	~50x faster than LLM-based extraction, ~50 MB model footprint
Rustworkx over NetworkX	`get_rustworkx_subgraph` in `graph.py`	C/Rust graph math is orders of magnitude faster than pure Python
Embedded Qdrant	In-process path mode (no separate server)	Zero network overhead, single-process memory management
Prompt Templates	`prompt_templates.py` (750 lines)	Few-shot examples tuned per node reduce hallucination in small models

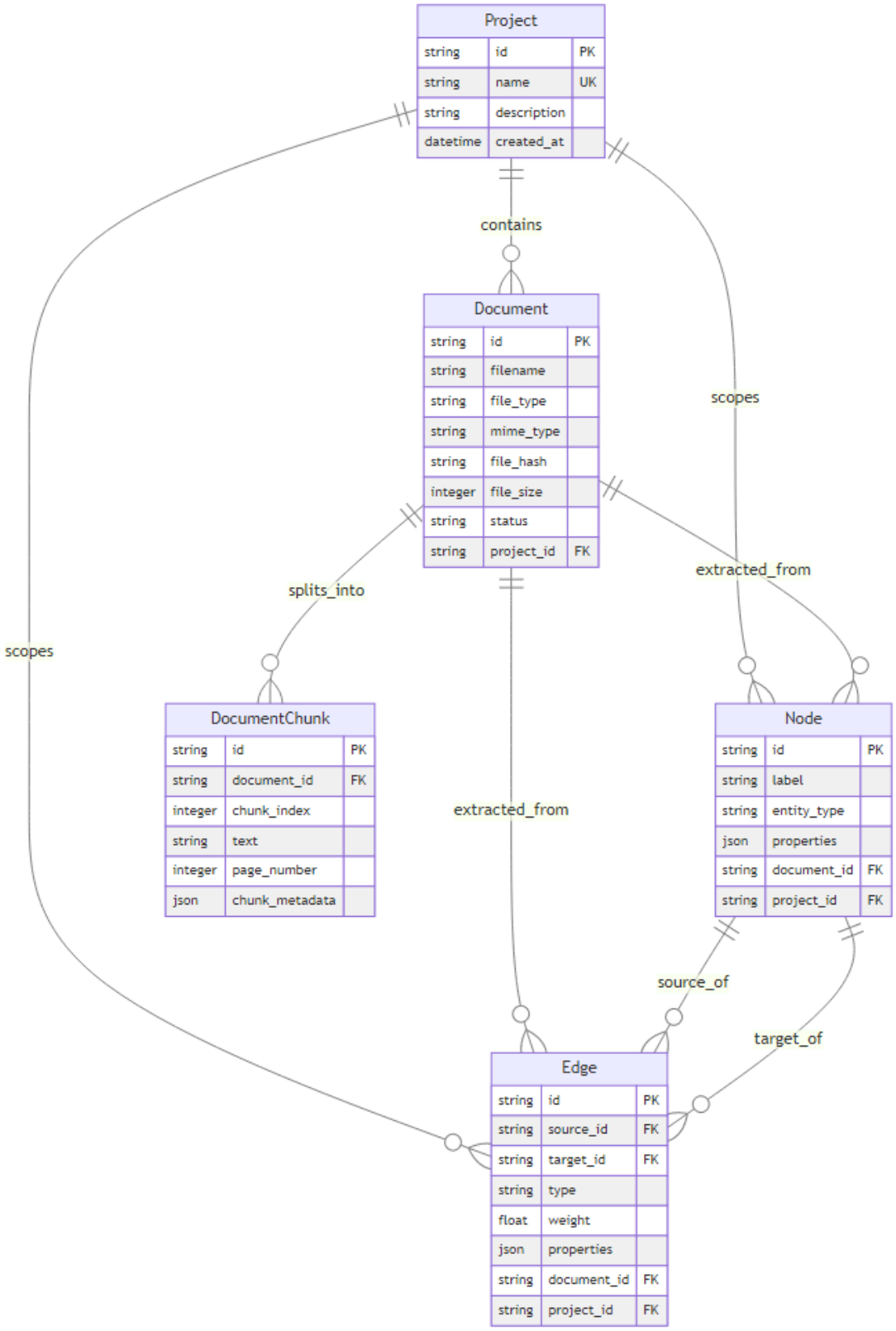
LLM Configuration (from `config.py`):

```
LLM_CONTEXT_SIZE    = 8192    # 8K context window (4096 for 3B if OOM)
LLM_N_BATCH         = 512     # Batch size for prompt processing
LLM_USE_MLOCK       = True    # Pin model weights in RAM
DEFAULT_GPU_LAYERS   = 35     # Partial VRAM offload
```

---

6. Data Model & Storage

All persistence is defined in `database.py`:



Key indexes (performance-critical): - `idx_nodes_label` — Fast entity lookup by type - `idx_edges_source_target` — Graph traversal queries - `idx_documents_file_hash` — Deduplication on upload - `idx_chunk_document` — Chunk retrieval by document

Graph edge ontology (from `GRAPH_ONTOLOGY_EDGE_TYPES` in config):

```
CAUSES, INHIBITS, ENABLES, PART_OF, RELATED_TO, CONTRADICTS,
SUPPORTS, CLINICAL_TRIAL_FOR, TREATS, DIAGNOSES, MEASURED_BY,
AUTHORED_BY, PUBLISHED_IN, FUNDED_BY
```

> [!NOTE] The edge types `CLINICAL_TRIAL_FOR`, `TREATS`, and `DIAGNOSES` are already built into the ontology — the codebase was designed with biomedical discovery in mind from day one.

---

7. Configuration & Extensibility

All 50+ parameters are defined in a single `Settings` class in `config.py`, loaded from `.env`:

Category	Parameters	Defaults
Navigator Reflection	`ENABLE_NAVIGATOR_REFLECTION`, `MAX_REFLECTION_ITERATIONS`, `NAVIGATOR_CONFIDENCE_THRESHOLD`	`True`, `3`, `0.75`
Cortex MoE	`ENABLE_CORTEX_CROSSCHECK`, `CORTEX_NUM_SUBTASKS`, `MOE_MAX_EXPERT_ROUNDS`, `MOE_HYPOTHESIS_COUNT`	`True`, `5`, `5`, `3`
RAG Pipeline	`ENABLE_RERANKING`, `RERANK_TOP_N`, `USE_SEMANTIC_CHUNKING`, `USE_RAPTOR`	`True`, `5`, `True`, `True`
Document Parsing	`USE_DOCLING`, `CHUNK_SIZE`, `CHUNK_OVERLAP`, `SEMANTIC_CHUNK_TOKENS`	`True`, `1000`, `200`, `512`
Graph	`ENABLE_EVIDENCE_BOUND_EXTRACTION`, `ENABLE_GRAPH_CRITIC`, `GRAPH_CACHE_TTL`	`True`, `True`, `300s`
Cloud Models	`DEEPSEEK_API_KEY`, `MINIMAX_API_KEY`, `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`	Empty (local-only)

---

8. The Vision: Discovery OS for Small Molecule & Glycan Research

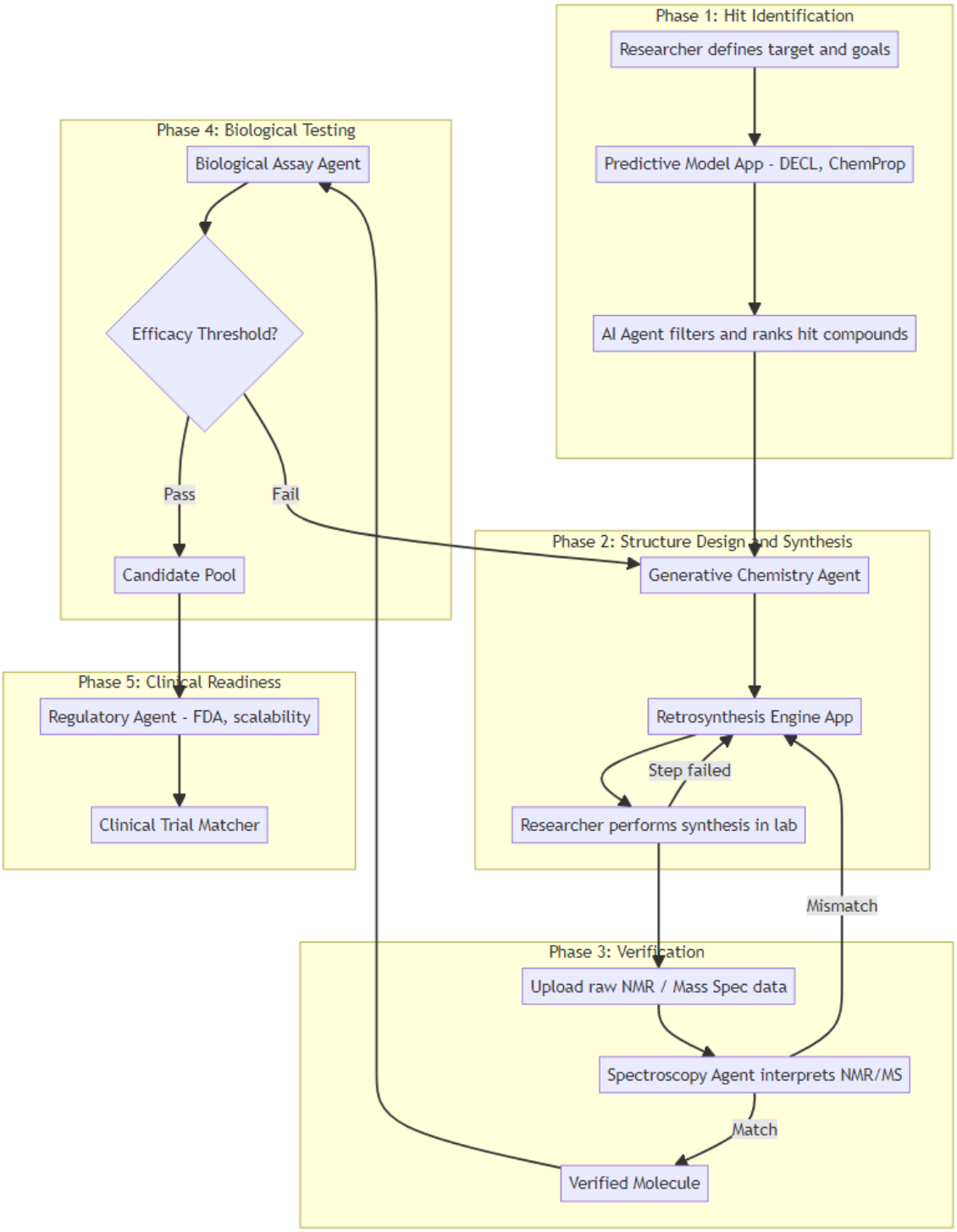
8.1 The Core Idea

Atlas today is a generalized research OS. The vision is to vertically integrate it into a **closed-loop drug discovery platform** — starting with one specific, high-value domain: **small molecule and glycan discovery**.

The philosophy: **Atlas is not a tool. It is an Operating System.**

Just as Windows provides the kernel and UI while third-party "apps" provide specialized functionality, Atlas provides the knowledge substrate, agent orchestration, and UI — while specialized **discovery apps** (retrosynthesis engines, NMR interpreters, assay analyzers) are plugged in as modular agents.

8.2 The Closed-Loop Discovery Workflow



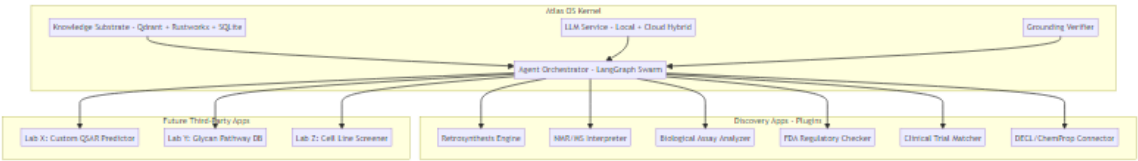
8.3 Why Glycans Are the "Killer App"

Molecule Class	Synthesis Automation	Atlas Opportunity
Peptides	Highly automated (SPPS machines)	Low — already solved
Nucleic Acids	Highly automated (phosphoramidite chemistry)	Low — already solved
Small Molecules	Semi-automated (HTS + medicinal chemistry)	Medium — AI can accelerate design
Glycans	Essentially manual; no universal template	**Extremely High** — unsolved problem

Glycans are critical in biological signaling, immune response, and disease markers — but their synthesis is prohibitively difficult due to: - **Non-linear branching** (unlike the linear chains of peptides/DNA) - **Complex stereochemistry** (anomeric configurations: α vs β glycosidic bonds) - **Protecting group orchestration** (dozens of orthogonal protection strategies)

> [!IMPORTANT] A researcher can spend **1–2 years synthesizing a single glycan molecule.** If Atlas can reliably predict retrosynthesis pathways for glycans, it is an immediate, fundamental breakthrough in biochemistry.

8.4 The Plugin Architecture



> [!TIP] The plugin system is the fundamental differentiator from Recursion, Schrodinger, and BenevolentAI — all of which are closed, vertically integrated pipelines. Atlas is the **open, local-first OS** where labs keep their IP entirely on their own machines.

---

9. Competitive Landscape, Moat, and Strategy

9.1 Competitor Analysis

The AI-for-science space is heavily funded but fragmented. Every competitor solves one slice of the R&D pipeline — none provides an open OS layer.

Company	Funding	What They Do	Weakness Atlas Exploits
Recursion Pharmaceuticals	$1.5B+ raised, public (RXRX)	Closed-loop phenomics + ML drug discovery	Fully closed platform. Your data is their data. No

Company	Funding	What They Do	Weakness Atlas Exploits
			plugin system. Only accessible to $B pharma budgets.
Schrödinger	Public (SDGR), $600M+ rev	Physics-based molecular simulation + ML	Licensing model ($100K+/yr per seat). Black-box solvers. No agentic AI, no knowledge graph.
Insilico Medicine	$400M+ raised	End-to-end AI drug discovery (target ID to clinical)	Proprietary pipeline — you use their targets, their chemistry. No local deployment.
BenevolentAI	Public (BAI), $400M+ raised	Knowledge graph for target discovery	Knowledge graph only — no synthesis planning, no NMR verification, no closed-loop feedback.
Relay Therapeutics	$700M+ raised	Motion-based drug design (protein dynamics)	Narrow focus on protein motion. No OS layer, no generalized R&D capability.
PostEra	$26M raised	ML for medicinal chemistry (Manifold platform)	Chemistry-only. No document intelligence, no knowledge graph, no multi-agent reasoning.
Chemify	$60M raised (Lee Cronin)	Automated chemical synthesis via “chemputation”	Hardware-dependent. Requires their proprietary robots. Not software-first.
Benchling	$6.1B valuation	Lab informatics and data management (ELN/LIMS)	Pure data management — zero AI reasoning. No synthesis planning. No agent intelligence.

The gap in the market: Every competitor above is either (a) a closed vertical pipeline you rent, or (b) a point solution for one step. **Nobody is building the open OS that connects all these steps with agentic intelligence.**

9.2 Why "Zero Lock-in" Is Not the Moat

Let's be honest: "your data stays local" and "zero lock-in" sound good in a pitch deck, but they are **not defensible**. Any competitor can claim the same thing tomorrow. If the only reason someone uses Atlas is that it's open, they'll leave the moment a closed product is 10% better.

The real question: **What makes Atlas impossible to leave once you're in?**

9.3 The Actual Moat: Compound Intelligence

Atlas wins by building three compounding advantages that get stronger with every user and every experiment:

Moat 1: The Data Flywheel (Synthesis Memory)

Every time a researcher uses Atlas to plan a synthesis and reports back what worked and what failed, that data trains the local retrosynthesis model. Over time, each lab's Atlas instance becomes a deeply personalized expert on *their specific chemistry*.

- A lab that has used Atlas for 2 years has 2 years of synthesis memory that no competitor can replicate.
- This is not transferable — it's embedded in their local knowledge graph.
- Switching to a competitor means starting from zero.

This is lock-in through *accumulated intelligence*, not through data hostage-taking.

Moat 2: The Plugin Ecosystem (App Store Effect)

If the best retrosynthesis plugin, the best NMR interpreter, and the best assay analyzer all run on Atlas, then leaving Atlas means losing access to those tools. This is the iOS/Android playbook:

- Apple doesn't lock you in by holding your photos hostage.
- Apple locks you in because the best apps are only on iOS.

Atlas's open plugin SDK is not a weakness — it's the mechanism that creates ecosystem lock-in. The more third-party tools built for Atlas, the harder it is to leave.

Moat 3: The Glycan Wedge (Domain Credibility)

Glycan synthesis is the single hardest unsolved problem in synthetic chemistry. If Atlas cracks it first — even partially — it earns instant, permanent credibility as the serious platform for hard chemistry. Every pharma company exploring glycan therapeutics (and there are many, because glycans are involved in cancer, autoimmune disease, and viral defense) will come to Atlas because nobody else has a working solution.

This is "Intel Inside" for chemistry: you don't switch away from the platform that solved the thing nobody else could.

9.4 Total Addressable Market: Every Company That Does R&D

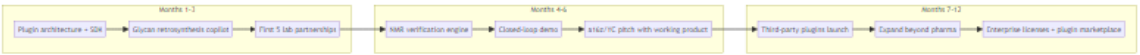
Targeting only biotech labs keeps Atlas small. But the core loop — **hypothesize → build → test → learn → iterate** — is not unique to chemistry. It is the universal loop of all research and development, everywhere.

ASML spends $4B/yr on R&D to engineer next-generation lithography machines. Apple spends $30B/yr designing the next iPhone. NVIDIA spends $10B/yr creating new chip architectures. F1 teams spend $150M+/yr developing aerodynamic packages. Every single one of these organizations has researchers drowning in documents, experimental data, and iterative design cycles that Atlas can accelerate.

Industry	Global R&D Spend	How Atlas Applies
Pharmaceuticals and Biotech	$250B/yr	Drug discovery, glycan therapeutics, biologics, clinical trial optimization
Semiconductors (ASML, NVIDIA, TSMC, Intel)	$150B/yr	Process node R&D, materials characterization, yield optimization, chip architecture iteration
Consumer Electronics (Apple, Samsung, Sony)	$80B/yr	Materials testing, component design iteration, failure analysis across product generations
Automotive and Motorsport (F1, Tesla, Toyota)	$120B/yr	Aerodynamics iteration, battery chemistry, powertrain development, CFD-to-wind-tunnel feedback loops
Aerospace and Defense (Lockheed, SpaceX, Boeing)	$100B/yr	Materials qualification, propulsion R&D, structural testing, compliance documentation
Materials Science	$50B/yr	Polymer design, catalyst discovery, alloy optimization, coating formulation
Battery and Energy	$30B/yr	Electrolyte design, solid-state materials, catalyst screening
Agricultural Chemistry	$15B/yr	Crop protection molecules, pesticide/herbicide design
Academic Research (all fields)	$700B/yr	Any lab doing experimental research with documents and iterative data

The total addressable market is not "biotech startups" — it is **every organization on Earth that iterates on experimental R&D**. That is **$2.5T+/yr** in global R&D spending. Atlas starts with pharma (highest pain, deepest domain complexity) and expands horizontally because the OS layer is domain-agnostic — only the plugins change.

9.5 How Atlas Wins: The Playbook



The sequence matters:

1. **Win the hardest problem first** (glycan synthesis) — this earns credibility that no amount of marketing can buy
2. **Ship the closed loop fast** — a working demo of hit → design → synthesize → verify is worth more than a perfect product
3. **Build the ecosystem** (plugin SDK) — switching costs compound with every third-party tool built on Atlas
4. **Expand horizontally** (materials, semiconductors, automotive) — the OS layer generalizes, only the plugins change
5. **Monetize the platform** (enterprise licenses + marketplace commission) — this is a $B revenue model, not a $M one

9.6 Feasibility Check: Algorithms and Robotics

Are current prediction algorithms good enough for AI to leverage?

Algorithm Type	Maturity	Atlas Strategy
Property Prediction (ChemProp, ADMET)	🟢 Production-ready	Wrap as plugin; AI agent invokes via API
Structure Prediction (AlphaFold 3, RoseTTAFold)	🟢 Production-ready for proteins	Integrate for target validation
Retrosynthesis (AiZynthFinder, ASKCOS)	🟡 Good for simple, weak on stereochem	Starting point — **human feedback loop is key**
NMR Prediction (nmrshiftdb2)	🟡 Databases exist; AI interpretation nascent	Train an NMR comparison agent
Glycan Pathways	🔴 Open-source is minimal	**The gap Atlas fills** — researcher-in-the-loop

Can robots handle synthesis?

Task	Robotic Readiness	Atlas Strategy
High-Throughput Screening	🟢 Mature (Opentrons, Hamilton)	Integrate plate reader data as input app
Organic Synthesis	🔴 Extremely difficult to roboticize	**Keep human-driven.** Atlas = brain, researcher = hands
Automated Flow Chemistry (ChemSpeed)	🟡 Specialized, expensive	Long-term integration target (Year 2+)

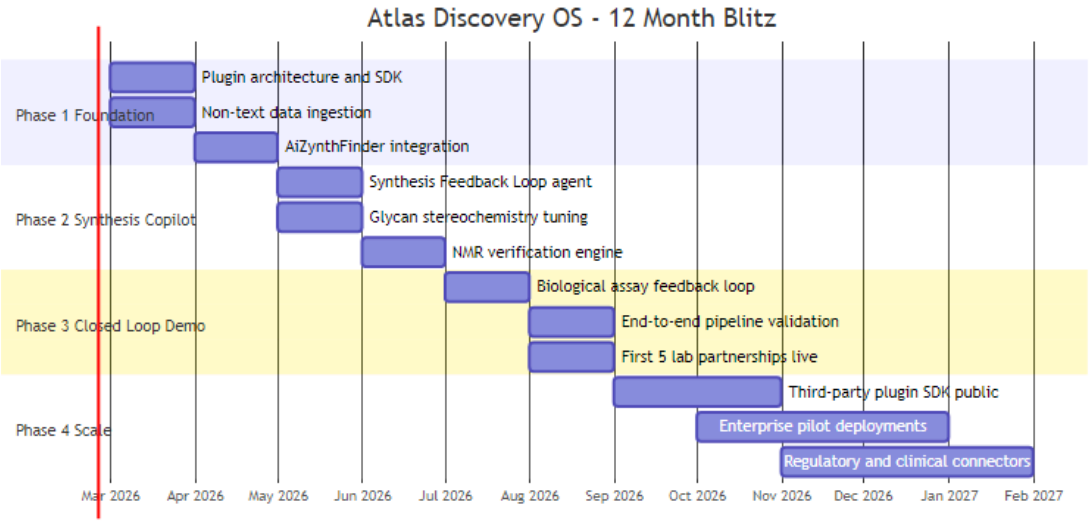
> [!NOTE] **Key insight:** Atlas's value is in the **informational bottleneck**, not the physical one. Design the pathway, interpret the NMR, decide what to screen next — these cognitive tasks are where AI creates 100x leverage. The wet lab stays

> human for now.

---

10. Roadmap

For a16z / YC Delivery — 12-Month Blitz



Phase Details

Phase	Timeline	Deliverables	Success Criteria
🧱 **1: Foundation**	Weeks 1–8	Plugin SDK, non-text data ingestion, AiZynthFinder integration	A Python plugin registers with Atlas and is invoked end-to-end
👷 **2: Copilot**	Weeks 8–16	Synthesis Feedback Loop, stereochemistry agent, NMR verification	Agent recalculates failed pathway in <60s; NMR confidence scored
🔄 **3: Closed Loop**	Weeks 16–24	Bio-assay feedback, pipeline validation, first 5 lab partners	Working demo: hit → synthesis → NMR → assay with zero manual transfer
🚀 **4: Scale**	Weeks 24–48	Public plugin SDK, enterprise pilots, FDA connectors	Third-party plugins shipped; enterprise revenue; FDA integration live

---

> *"Atlas is not another AI tool. It is the operating system for R&D — starting with the hardest molecule class on Earth, then expanding to every industry that builds the future."*